PECS Communication App

Initial Design Document

Document Version: 0.1
Project Stage: Concept / MVP Planning

Contents

1. Project Overview 3
2. Primary Design Goals 4
   2.1 Simple Communication 4
   2.2 Minimal Distraction 4
3. Initial Target 6
4. Initial Card Set 6
5. Card Data Model 7
6. Media Asset Architecture 9
7. Copyright and Asset Replacement 10
8. Animation Strategy 12
   8.1 Card Animation 12
   8.2 Interface Animation 12
9. Preferred Animation Technology 13
10. MVP User Interface 14
11. Exchange Interaction 16
      Method A - Tap 16
      Method B - Drag to Exchange 16
12. Exchange Confirmation 17
13. Audio 18
14. Exchange Logging 18
15. Offline-First Architecture 19
16. Parent / Configuration Mode 20

17. Custom Cards 21
18. Categories 22
19. Phrase Builder 22
20. Daily Schedule 23
21. Social Stories 24
22. Multi-User Profiles 24
23. Two-Device Mode 25
24. Accessibility 26
25. Privacy 27
26. Technical Architecture 27
27. Suggested Core Services 28
28. Local Database 29
29. Suggested Project Development Stages 30
    Stage 1 - Card Prototype 30
    Stage 2 - Digital Exchange 30
    Stage 3 - Audio 30
    Stage 4 - History 31
    Stage 5 - Custom Cards 31
    Stage 6 - Animation 31
30. Recommended MVP Scope 32
31. Explicitly Outside the MVP 33
32. Long-Term Feature Progression 33
33. Guiding Architectural Principle 34
34. Initial Success Criteria 35

1. Project Overview

The PECS Communication App is an offline-first communication application inspired by the Picture Exchange Communication System.

The application provides a simple digital interface in which a user selects or exchanges visual cards representing desired objects, actions, needs, or concepts.

The initial application will focus on the simplest form of communication:

Select a picture card and give it to a communication partner.

The project will intentionally begin with a very small card library and limited functionality. Additional communication capabilities will be introduced incrementally after the basic interaction model has been tested and stabilized.

The long-term system should support:

- Individual picture exchanges
- Phrase construction
- Personalized cards
- Personal photographs
- Custom voice recordings
- Animated cards
- Daily visual schedules
- Communication history and statistics
- Multiple user profiles
- Communication between two devices
- Parent/teacher configuration tools

The application should remain usable without an internet connection.

---

2. Primary Design Goals

2.1 Simple Communication

The interface should require as few actions as possible.

A child should be able to:

1. Find the desired card.
2. Select or drag the card.
3. Exchange it with the communication partner.
4. Receive immediate confirmation.

Navigation, settings, menus, and configuration controls should not interfere with this interaction.

2.2 Minimal Distraction

The application is a communication tool rather than a game.

Visual effects should therefore be purposeful.

Animations may be used to:

- Demonstrate an action represented by a card.
- Show that a card has been exchanged.
- Draw attention to the communication partner.
- Reinforce the meaning of an action.

Animations should not exist purely as decorative rewards.

The user interface should avoid:

- Excessive movement
- Animated backgrounds
- Particle effects
- Unnecessary sounds
- Complex menus

- Small buttons
- Advertising

2.3 Multilingual from the Start

The application must support multiple interface languages from the earliest prototype.

The initial supported languages are:

- English
- Russian

All user-facing text must be localized. No user-facing strings should be hard-coded directly into application screens or business logic.

This includes:

- Navigation labels
- Buttons
- Settings
- Card labels
- Confirmation messages
- Error messages
- Parent-mode interfaces
- Statistics labels
- Accessibility options
- Dialogs and prompts

The application settings must include a language selector.

Changing the application language should update the interface without requiring the user to reinstall the application or download a different version.

The localization architecture must allow additional languages to be added later without redesigning the application.

---

3. Initial Target

The initial version will target users beginning with simple picture exchanges corresponding approximately to early PECS usage.

The MVP will use:

- One device (Android 13+ and Windows 11 versions, MacOS and iOS will be a future update)
- One child/user profile
- One communication screen
- Approximately 10 cards
- Still images
- Optional spoken confirmation
- Local exchange history
- Offline operation

The MVP will **not** attempt to implement the entire PECS methodology.

Its purpose is to establish the underlying application architecture and determine whether the digital exchange interaction is useful and intuitive.

---

4. Initial Card Set

The first release should contain approximately 10 cards.

The precise cards can change during testing, but the initial collection should represent extremely common requests.

Example set:

1. Water
2. Food
3. Apple
4. Toilet

5. Help
6. More
7. Stop
8. Play
9. Break
10. Finished

The exact starting vocabulary should eventually be reviewed with people familiar with the intended users.

Cards should not be hard-coded into the UI.

They should be loaded from a card definition system so that cards can be added, removed, renamed, translated, or have their media replaced without changing application code.

---

5. Card Data Model

Each communication card should be represented as data.

A conceptual card object may contain:

Card

```
{
    Id
    Name
    DisplayText
    Category
    Image
    Animation
    Audio
    VoiceText
    Enabled
    SortOrder
```

```
}
```

A more complete future model could contain:

```
Card
{
    Id: UUID

    Name: string
    DisplayText: string

    CategoryId: UUID?

    Media:
    {
        ImageAssetId
        AnimationAssetId
        ThumbnailAssetId
    }

    Audio:
    {
        AudioAssetId
        TextToSpeechText
    }

    Behavior:
    {
        ExchangeEnabled
        PhraseBuilderEnabled
        ScheduleEnabled
```

```
    }

    Metadata:
    {
        CreatedDate
        UpdatedDate
        Source
        IsCustom
    }
}
```

The UI should reference the card's ID rather than directly referencing an image filename.

---

6. Media Asset Architecture

Replaceable media is an important architectural requirement.

Images should **not** be tightly coupled to individual screens or application code.

Instead, media should be treated as assets.

Example:

```
Card
    |
    +-- ImageAssetId
    +-- AnimationAssetId
    +-- AudioAssetId
```

An asset table or manifest resolves these references.

Example:

```
MediaAsset
{
    Id
```

```
    Type
    FilePath
    Format
    Version
}
```

Supported media types may eventually include:

`StillImage`

`AnimatedImage`

`VectorAnimation`

`Video`

`Audio`

This allows a card such as:

`apple`

to initially use:

`apple-v1.png`

and later be updated to:

`apple-v2.webp`

or:

`apple-animation-v1.json`

without changing the application's card logic.

---

7. Copyright and Asset Replacement

The application should assume that **every bundled visual asset may eventually need to be replaced**.

Reasons may include:

- Copyright concerns
- Licensing changes

- Inconsistent visual style
- Improved artwork
- Accessibility improvements
- Replacement with custom artwork
- Replacement with animation

For this reason, an application's business logic must never depend upon the visual representation of a card.

For example:

```
Card ID:
3ec3...

Concept:
Apple

Current visual:
apple-temp.png
```

Later:

```
Card ID:
3ec3...

Concept:
Apple

Current visual:
apple-custom-animation.json
```

The card remains the same logical card despite the media changing.

This also allows application updates to replace an asset without destroying:

- User history

- Card preferences
- Usage statistics
- Phrase history
- Custom sorting

---

8. Animation Strategy

Animations should be introduced after the still-image system is stable.

The application should support two distinct animation concepts.

8.1 Card Animation

The image itself explains the concept.

Examples:

Drink

A character raises a cup and drinks.

Eat

A character takes food to their mouth.

Sleep

A character lies down and closes their eyes.

Wash Hands

A short looping handwashing animation is displayed.

These animations may eventually replace static illustrations.

8.2 Interface Animation

These animations explain something happening inside the application.

Examples:

- Card moves toward the communication partner.

- Partner's hand accepts the card.
- Sentence cards slide into the sentence bar.
- Completed schedule item moves into a "Finished" area.

These animations should remain short and predictable.

---

9. Preferred Animation Technology

Animations should ideally use a format that is lightweight and independent from application logic.

Possible formats include:

- Lottie
- Rive
- Animated WebP
- Short MP4/WebM clips

For custom vector-based animations, **Rive or Lottie** are particularly attractive because they provide scalable graphics without storing large video files.

The media abstraction should prevent the rest of the application from depending upon a particular animation technology.

For example:

`IMediaRenderer`

could determine whether an asset should be displayed using:

`StaticImageRenderer`

`LottieRenderer`

`RiveRenderer`

`VideoRenderer`

This lets the application's animation technology change later without redesigning the card system.

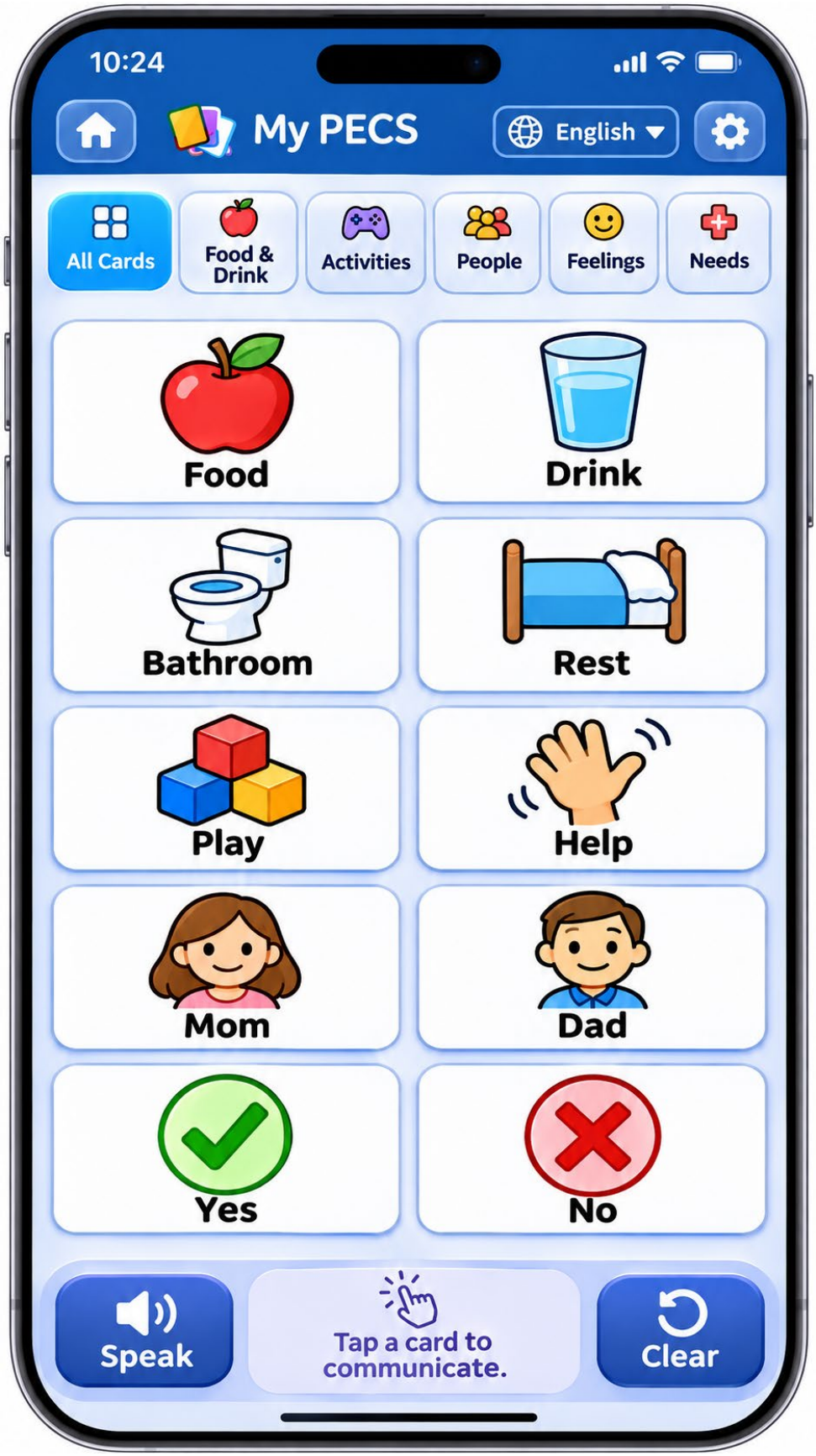
---

10. MVP User Interface

The initial user experience should contain only a few major areas.

Main Communication Screen

Example phone layout:


10:24
My PECS
English
All Cards
Food & Drink
Activities
People
Feelings
Needs
Food
Drink
Bathroom
Rest
Play
Help
Mom
Dad
Yes
No
Speak
Tap a card to communicate.
Clear

Card images should be large enough to recognize the images easily.

The number of cards shown simultaneously may eventually become configurable with fewer cards ideal for lower resolution and smaller screens and more cards working well in the desktop versions of the app.

---

11. Exchange Interaction

Two interaction methods should eventually be supported.

Method A - Tap

The user taps a card.

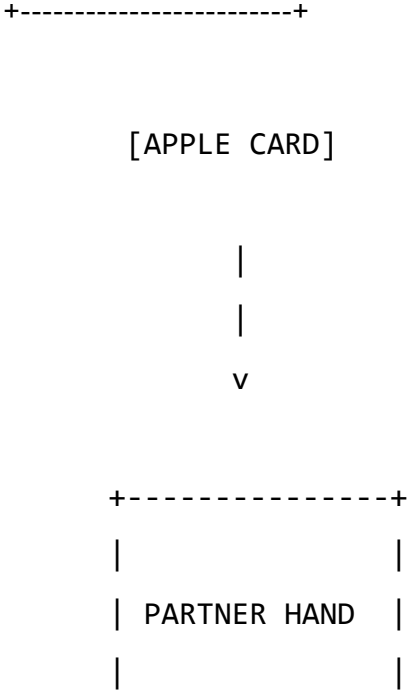
The application immediately communicates the request.

This is the easiest accessibility option.

Method B - Drag to Exchange

The user drags the card to a target representing the communication partner.

Example:



```
        +---------------+


+-------------------------+
```

When the card reaches the exchange target:

1. The card is accepted.
2. A short animation plays.
3. The request is spoken.
4. The exchange is recorded.

Example:

"I want an apple."

This interaction more closely resembles the physical act of exchanging a PECS card.

Both input methods should eventually be configurable.

---

12. Exchange Confirmation

After an exchange, feedback should be immediate but simple.

Possible feedback:

- Spoken phrase
- Short acceptance animation
- Brief card enlargement
- Visual confirmation

Example:

`APPLE`

becomes:

```
You asked for:


APPLE
```

while audio says:

"Apple."

or:

"I want an apple."

The parent should eventually be able to configure this behavior.

---

13. Audio

Every card should support three audio modes.

Built-In Voice

The device's text-to-speech engine reads the card.

Example:

```
TextToSpeechText = "I want water."
```

Bundled Audio

A prerecorded application audio file is played.

Personal Recording

A parent records a familiar voice.

Example:

"Water."

or:

"I want water."

Personal recordings should be stored locally and associated with the card.

---

14. Exchange Logging

Every completed exchange should create a local history record.

Example:

```
Exchange
{
    Id
    CardId
    UserId
    Timestamp
    InteractionMethod
}
```

For example:

```
2026-10-29 08:12 - Water
2026-10-29 08:47 - Food
2026-10-29 09:14 - Play
2026-10-29 10:32 - Water
```

The MVP does not require complicated analytics.

The first statistics screen may only show:

```
Today


Water      5
Food       3
Play       3
Help       2
Break      1
```

The raw event history should still be stored so more advanced statistics can be developed later.

---

15. Offline-First Architecture

All essential communication functionality must work without internet access.

Locally stored information includes:

- Cards
- Images
- Animations
- Audio
- Personal recordings
- Settings
- User profile
- Exchange history

Internet connectivity should only become necessary for optional future capabilities such as:

- Cloud synchronization
- Downloading additional card packs
- Remote backups
- Two-device communication outside a local network

Lack of internet access must never prevent someone from using their communication cards.

---

16. Parent / Configuration Mode

Configuration controls should not appear within the normal communication interface.

A separate Parent Mode should eventually provide:

```
Cards
Profiles
Audio
History
Statistics
Settings
```

Entering Parent Mode could require:

- PIN
- Password
- Long press
- Device authentication

This avoids accidental configuration changes.

---

17. Custom Cards

Custom cards should become one of the first major features after the MVP.

Parents should be able to:

1. Take a photograph.
2. Choose an existing photograph.
3. Enter the card name.
4. Record a voice clip.
5. Assign a category.
6. Add the card to the communication board.

Example:

```
Card Name:
Blue Cup

Image:
[Photo of child's actual blue cup]

Voice:
"I want my blue cup."
```

Using photographs of real objects can make the application substantially more personalized.

---

18. Categories

The MVP does not necessarily require categories.

However, the architecture should support them immediately.

Possible categories include:

```
Food

Drinks

Toys

Activities

People

Places

Needs

Feelings

Actions

School
```

A card may belong to one or more categories in the future.

---

19. Phrase Builder

Phrase construction should be a later development stage.

Example interface:

```
+-------------------------------------+


Sentence:


[I WANT] [APPLE]
```

```
+-------------------------------------+

Cards:

[APPLE] [WATER] [TOY] [BREAK]

+-------------------------------------+
```

Selecting the sentence bar would produce:

"I want an apple."

Future starter phrases could include:

```
I want...

I need...

I see...

I feel...

I like...

I don't want...
```

This should reuse the existing card system rather than introduce a second type of communication object.

---

20. Daily Schedule

A later schedule system can also reuse communication cards.

Example:

```
MORNING

[Breakfast] -> [Brush Teeth] -> [Get Dressed] -> [School]
```

Completed activities could move into:

```
FINISHED
```

or receive a visible check mark.

Schedule functionality should be implemented as a separate feature using the same underlying Card entities.

---

21. Social Stories

Social stories should be implemented after the primary communication features are mature.

A story could consist of a sequence of:

```
StoryStep
{
    Prompt
    Image
    AvailableCardChoices
    CorrectResponse
}
```

Example:

```
Someone gives you a toy.

What can you say?

[THANK YOU]
[STOP]
[FOOD]
```

Feedback should remain gentle and non-punitive.

---

22. Multi-User Profiles

Future versions may support multiple communication profiles.

Example:

```
Profile
{
    Id
    Name
    SelectedCards
    PreferredVoice
    InteractionMode
    PhraseLevel
}
```

Each profile could independently configure:

- Available cards
- Card order
- Categories
- Recorded voice
- Communication level
- Schedule
- Accessibility settings

---

23. Two-Device Mode

A later version may allow two connected devices.

Example:

```
Child Tablet
     |
     | Local Wi-Fi / Bluetooth
     v
Parent Phone
```

The child exchanges:

`WATER`

The parent's device displays:

`Water requested`

and optionally plays a notification.

This should not be part of the initial MVP because networking adds substantial complexity and failure conditions to a feature that needs to remain extremely reliable.

---

24. Accessibility

Accessibility must be treated as a core architectural concern rather than a later enhancement.

Potential settings include:

- Card size
- Number of cards per screen
- Text visibility
- High-contrast mode
- Animation enabled/disabled
- Animation speed
- Audio volume
- Tap interaction
- Drag interaction
- Hold duration
- Confirmation behavior

The app should avoid relying exclusively on:

- Color
- Small text

- Fine motor movement
- Complex gestures

---

25. Privacy

The application may eventually contain highly personal information including:

- Children's photographs
- Voice recordings
- Communication history
- Behavioral patterns
- Daily schedules

The default design should therefore keep this information on the device.

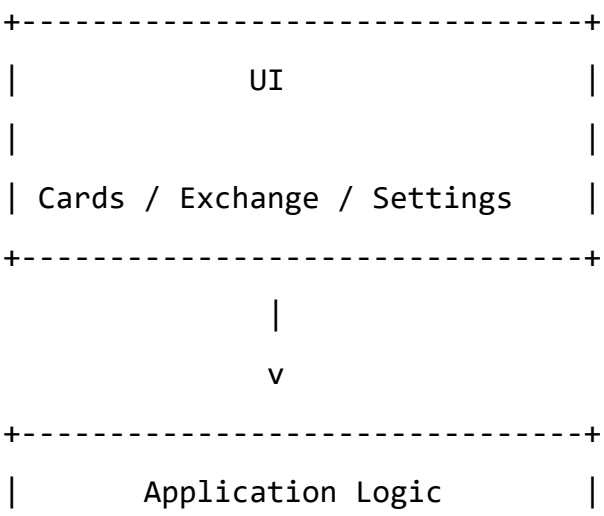
Cloud services should be optional.

No analytics, advertising SDKs, or third-party tracking should be required for the communication functionality.

---

26. Technical Architecture

A layered architecture is recommended.



```
|                                 |
| ExchangeService                 |
| CardService                     |
| AudioService                    |
| MediaService                    |
| LoggingService                  |
+---------------------------------+
                |
                v
+---------------------------------+
|              Data               |
|                                 |
| SQLite                          |
| Local Media                     |
| Preferences                     |
+---------------------------------+
```

This keeps the interface separate from card storage and media implementation.

---

27. Suggested Core Services

The application's business layer may eventually contain services such as:

`CardService`

`ExchangeService`

`MediaService`

`AudioService`

`HistoryService`

`ProfileService`

`SettingsService`

Later additions:

`ScheduleService`

`PhraseService`

`SyncService`

`StoryService`

`StatisticsService`

---

28. Local Database

SQLite is suitable for local structured application data.

Possible initial tables:

`Cards`

`MediaAssets`

`Categories`

`CardCategories`

`ExchangeHistory`

`Settings`

Later:

`Profiles`

`ProfileCards`

`Phrases`

`Schedules`

`ScheduleItems`

`Stories`

`StorySteps`

`VoiceRecordings`

Media files themselves should generally remain files rather than binary database objects.

The database stores references to those files.

---

29. Suggested Project Development Stages

Stage 1 - Card Prototype

Goal:

Display the initial 10 cards.

Features:

- Card data model
- Replaceable image assets
- Card grid
- Basic tap interaction

No logging or audio is required yet.

Stage 2 - Digital Exchange

Add:

- Drag-and-drop
- Exchange target
- Confirmation animation
- Exchange completion event

At this point, the application's core communication interaction exists.

Stage 3 - Audio

Add:

- Text-to-speech
- Card audio configuration
- Volume settings

Then add personal voice recording.

Stage 4 - History

Add:

- SQLite
- Exchange event storage
- Simple history page
- Daily request counts

Stage 5 - Custom Cards

Add:

- Import photograph
- Camera photograph
- Card creation
- Custom voice recording
- Card enable/disable
- Card ordering

At this stage, the application becomes useful beyond the bundled demo vocabulary.

Stage 6 - Animation

Replace selected still images with custom animations.

Test:

- Performance
- Battery usage
- Visual distraction
- File size
- Animation clarity

This phase establishes the final visual style before producing a large animation library.

30. Recommended MVP Scope

The first publicly usable version should contain:

Communication

- 10 bundled cards
- Large card grid
- Tap-to-request
- Drag-to-exchange
- Exchange target
- Visual confirmation
- Spoken confirmation

Data

- Local SQLite storage
- Exchange history
- Basic card configuration

Media

- Replaceable images
- Media abstraction layer
- Audio support

Parent Controls

- Parent/settings area
- Enable or disable cards
- Change card order

System

- Fully offline operation

- No account required
- No advertisements
- No mandatory cloud services

---

31. Explicitly Outside the MVP

The following features should **not** be included in the initial development milestone:

- Cloud accounts
- Cloud synchronization
- Two-device mode
- Bluetooth communication
- Daily schedules
- Social stories
- Phrase builder
- Multiple profiles
- Downloadable card marketplace
- Complex statistics
- AI-generated cards
- Large card libraries

These are desirable future features, but including them during initial development would make it substantially harder to validate and stabilize the core exchange interaction.

---

32. Long-Term Feature Progression

A practical progression would be:

```
Digital Exchange
```

```
       ↓
Custom Cards
       ↓
Phrase Builder
       ↓
Daily Schedule
       ↓
Multiple Profiles
       ↓
Parent Analytics
       ↓
Two-Device Exchange
       ↓
Social Stories
```

Each stage builds upon the same Card, Media, Audio, Profile, and History architecture.

---

33. Guiding Architectural Principle

The most important architectural rule for the project should be:

A card represents a communication concept, not an image.

The card:

`"Drink"`

may be represented today by:

`drink.png`

next year by:

`drink-animation.riv`

and for a particular child by:

`photo-of-my-cup.jpg`

All three represent the same underlying communication concept.

Separating the **concept** from the **representation** allows the application to evolve without continually rebuilding its communication system.

---

34. Initial Success Criteria

The first prototype can be considered successful when a user can:

1. Open the application.
2. Immediately see approximately 10 communication cards.
3. Identify the desired card.
4. Tap or drag the card to exchange it.
5. Receive clear audio/visual confirmation.
6. Perform another exchange without navigating through menus.
7. Use the entire process without internet access.

Everything beyond these seven behaviors is secondary during the first implementation.